

Network access controls — security groups, endpoint policies, and the layers between

Companion to [architecture.md](#), answering one question in depth: **who is allowed to talk to what, and which mechanism enforces it**. Three visuals carry the story; the tables give the exact rules. Diagrams are generated by [diagrams/generate.py](#) — regenerate rather than hand-editing the SVGs.

Accepted risks, up front (SSP-scoped decisions, not oversights):

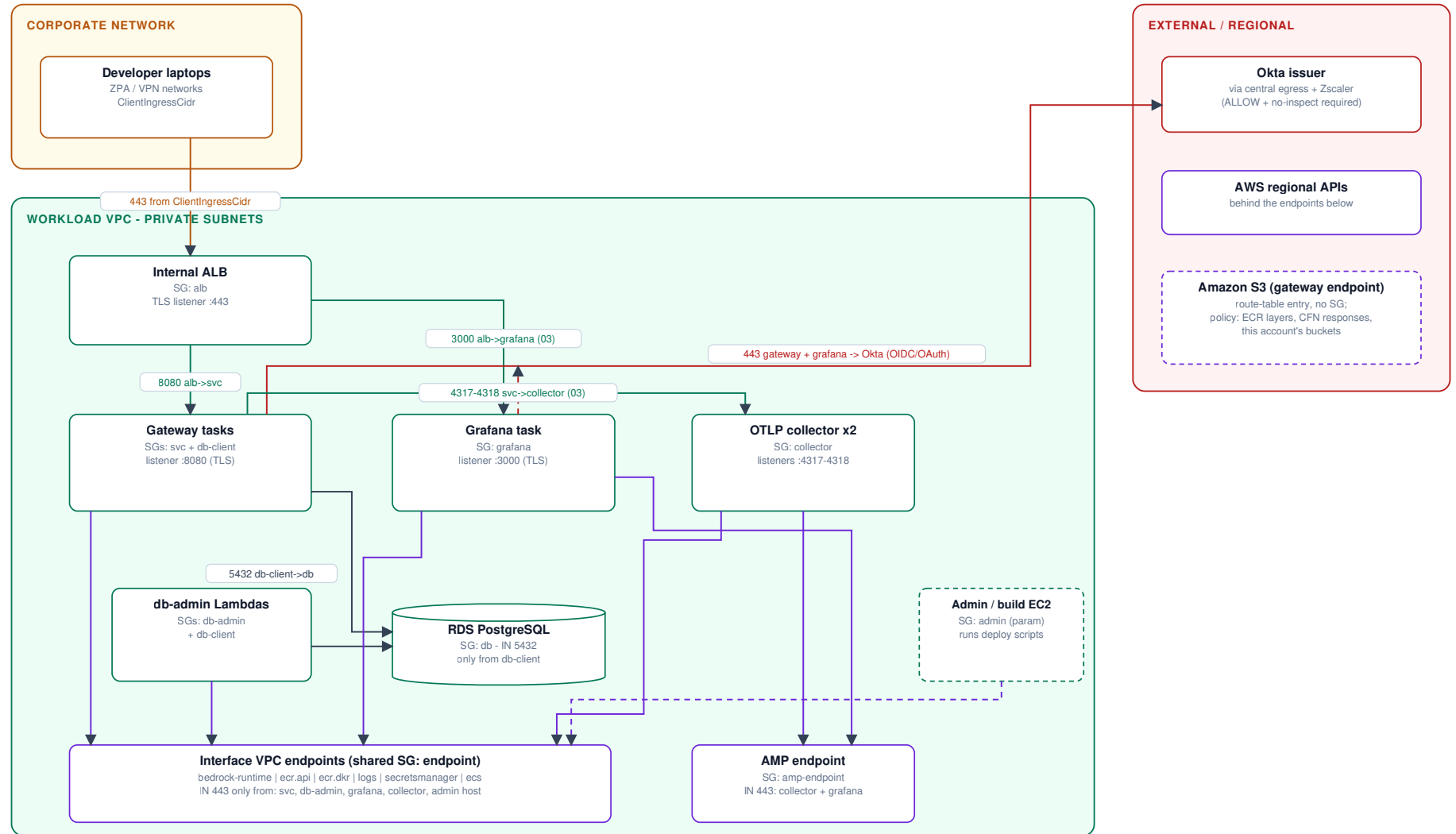
- The gateway→collector OTLP hop (4317–4318) is **plaintext**, compensated by SG-to-SG scoping — only gateway tasks can reach the collector ports (security review C2; the TLS recipe is documented on the collector task).
 - The `ecs` interface endpoint carries **no endpoint policy** — GovCloud does not support one there; IAM-side scoping covers it.
 - The ALB access-logs bucket is SSE-S3, not CMK — ELB log delivery does not support KMS.
-

1. The map — who may talk to whom

Every arrow is an explicit SG rule *pair*: an egress rule on the source group **and** an ingress rule on the destination group. A connection with only one half configured does not work; a connection with neither is dropped silently. Every group suppresses default egress with a `127.0.0.1/32` placeholder, so nothing here is reachable "by default."

7. Security groups - who may talk to whom

Every arrow is an explicit SG rule pair (egress on the source + ingress on the destination). Everything else is denied.



Not shown: every SG's default egress is a 127.0.0.1/32 placeholder (deny) unless a rule above exists; endpoint ENIs and RDS initiate nothing. Dashed = optional/conditional (admin-host param, AMP endpoint toggle, Grafana's Okta hop).

Reading tips:

- **Workloads carry more than one SG.** The gateway tasks run with `svc + db-client`; the db-admin Lambdas with `db-admin + db-client`. Attaching 01's `db-client` group is *the* mechanism for "may reach the database" — nothing else is ever granted 5432.
- **The interface endpoints are the chokepoint.** Their private DNS captures AWS API calls from **every** VPC client, so any in-VPC caller of `ecr/logs/secretsmanager/ecs` — including the admin/build host — must appear in the endpoint SG's ingress or its calls black-hole (this bit the first test run twice).
- **Okta is the only external dependency** the workloads dial out to; it rides the landing zone's central egress and needs the Zscaler ALLOW + no-inspect prerequisite (see `networking-request-email.md`).

2. The inventory — every rule, by group

8. Security-group rule inventory

Every rule in the deployment, by group. IN = ingress, OUT = egress. Groups not listed here do not exist.

alb (internal ALB)

02

IN 443 from ClientIngressCidr (developer networks)
OUT 8080 to svc (forward + health checks)
OUT 3000 to grafana (rule added by 03)

db-client (attach to reach the DB)

01

IN none
OUT 5432 to db
attached to: gateway tasks, both db-admin Lambdas

svc (gateway tasks)

02

IN 8080 from alb
OUT 443 0.0.0.0/0 (Okta, AWS APIs via endpoints/egress)
OUT proxy port 0.0.0.0/0 (only when HttpsProxyUrl set)
OUT 4317-4318 to collector (rule added by 03)
also carries db-client (below) for 5432

db (RDS instance)

01

IN 5432 from db-client
OUT none

collector (OTLP collector tasks)

03

IN 4317-4318 from svc (gateway OTLP forward)
OUT 443 0.0.0.0/0 (AMP remote_write, ECR, logs)

db-admin (bootstrap + rotation Lambdas)

02

IN none
OUT 443 0.0.0.0/0 (Secrets Manager, ECS via endpoints)
also carries db-client (below) for 5432

grafana (dashboard task)

03

IN 3000 from alb (path rule /grafana)
OUT 443 0.0.0.0/0 (AMP queries, Okta OAuth, ECR)

endpoint (shared, all 02 interface endpoints)

02

IN 443 from svc, db-admin
IN 443 from collector, grafana (rules added by 03)
IN 443 from AdminClientSecurityGroup (optional param)
OUT none (endpoint ENIs never initiate)

amp-endpoint (aps-workspaces endpoint)

03

IN 443 from collector (remote_write)
IN 443 from grafana (queries)
OUT none

Cross-stack rule writers

03 adds rules to SGs it imports from 02:
- AlbToGrafanaEgress: alb OUT 3000 -> grafana
- GatewayToCollectorEgress: svc OUT 4317-18 -> collector
- CollectorToEndpointsIngress: endpoint IN 443
- GrafanaToEndpointsIngress: endpoint IN 443
02 param AdminClientSecurityGroupId:
- AdminToEndpointsIngress: endpoint IN 443

Deploy order matters: the imported SGs must exist
(02 before 03), and CREATE_SUPPORTING_ENDPOINTS
must match between the two deploys.

Reading the map

A connection works only when BOTH ends agree:
an egress rule on the source SG and an ingress
rule on the destination SG. Every SG here is
default-deny; the 127.0.0.1/32 egress entries in
the templates are deliberate no-op placeholders
that suppress the default allow-all egress.

Accepted risk (SSP-scoped): the gateway->collector OTLP hop (4317-4318) is plaintext, compensated by SG-to-SG scoping - only gateway tasks can reach the collector ports. See security-review C2.

SG	Stack	Attached to	Ingress	Egress
alb	02	internal ALB	443 from ClientIngressCidr	8080→ svc ; 3000→ grafana (03)
svc	02	gateway tasks	8080 from alb	443 anywhere; proxy port (optional); 4317-18→ collector (03)
db-client	01	gateway tasks, db-admin Lambdas	—	5432→ db
db	01	RDS instance	5432 from db-client	—
db-admin	02	bootstrap + rotation Lambdas	—	443 anywhere
endpoint	02	all 02 interface endpoints	443 from svc , db-admin , collector (03), grafana (03), admin host (param)	—
collector	03	OTLP collector tasks	4317-18 from svc	443 anywhere
grafana	03	Grafana task	3000 from alb	443 anywhere
amp- endpoint	03	aps-workspaces endpoint	443 from collector , grafana	—

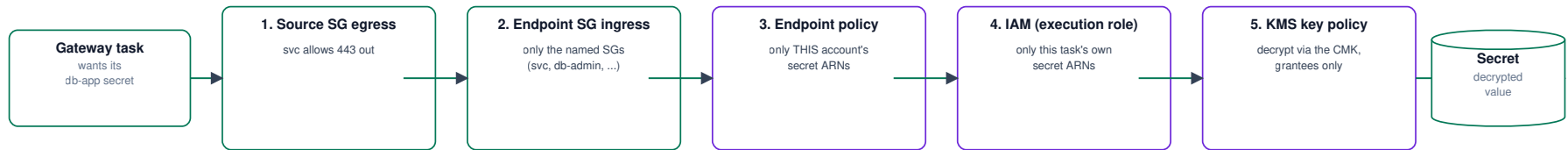
Cross-stack rule writers (03 and the admin-host parameter modify imported 02 SGs as separate `SecurityGroup{In,E}gress` resources): `AlbToGrafanaEgress` , `GatewayToCollectorEgress` , `CollectorToEndpointsIngress` , `GrafanaToEndpointsIngress` , `AdminToEndpointsIngress` . Deploy order (02 before 03) and a matching `CREATE_SUPPORTING_ENDPOINTS` across both deploys are what make these land correctly.

3. The layers — what each control stops that the others can't

SGs are one of **five** gates on the path of a single AWS API call. The diagram traces a gateway task reading its DB secret; the failure table under it is the reason none of the layers is redundant — the second row (attacker-owned credentials exfiltrating *through* a private endpoint) is the one SGs and your own IAM cannot stop alone, and is why every endpoint carries a resource policy.

9. Layered access control - what each layer stops

One request traced through every gate: a gateway task reading its DB secret through the Secrets Manager endpoint.



Why the layers are not redundant - each stops a different failure:

- **Compromised box in another subnet, ANY credentials**
stopped at layer 2 - its SG is not in the endpoint SG ingress (network path denied before authn)
- **Compromised workload using ATTACKER-OWNED AWS credentials (exfiltration through the private endpoint)**
stopped at layer 3 - the endpoint policy allows only this account's resources; your IAM cannot judge foreign principals
- **Legitimate workload asking for a secret it does not own**
stopped at layer 4 - execution roles enumerate exact ARNs (gateway task cannot read the master secret: no task injects it)
- **Principal with a stolen ciphertext or over-broad S3/API access**
stopped at layer 5 - the CMK key policy grants decrypt only to the roles that need it

Non-SG enforcement points elsewhere in the deployment

bedrock-runtime endpoint policy: pinned to the TWO configured model IDs + their inference profiles (not anthropic.*) | ecs endpoint: NO policy (GovCloud unsupported) - IAM-side scoping covers it
S3 gateway endpoint policy: ECR layer bucket + CloudFormation custom-resource response buckets + this account's buckets only
ALB access-log bucket policy: exactly two writers (ELB log-delivery service principal + legacy regional ELB account) | AMP endpoint policy: this workspace's ARN only
Okta egress: Zscaler policy must ALLOW + not inspect the issuer FQDN for server-originated traffic (org prerequisite - see the networking request)

Endpoint/resource policies in force:

Where	Policy scope
bedrock-runtime endpoint	the two configured model IDs + their inference profiles — not anthropic.*
ecr.api / ecr.dkr endpoints	auth token (no-resource) + this account's repositories
logs endpoint	this account's log groups
secretsmanager endpoint	this deployment's secret ARNs + GetRandomPassword
ecs endpoint	none (unsupported in GovCloud) — IAM covers
S3 gateway endpoint	ECR layer bucket, CloudFormation custom-resource response buckets, this account's buckets
aps-workspaces endpoint (03)	this AMP workspace's ARN only
ALB access-logs bucket	exactly two writers: ELB log-delivery service principal + legacy regional ELB account

IAM and KMS (layers 4–5) are inventoried in `architecture.md` §6 — the short version: execution roles enumerate exact secret ARNs, the task role holds `bedrock:InvokeModel*` on exactly the two models, and the single CMK's key policy names its grantees.

Change discipline: a new workload that needs the database gets `db-client` attached (never a new 5432 rule); a new in-VPC caller of AWS APIs gets an ingress on the endpoint SG (or it will black-hole); anything touching the endpoint SG or the observability SGs should re-check the cross-stack reachability findings in `security-review-2026-07.md`.